Carnegie Mellon University Africa

Name: Martin Koome

Andrew ID: mkoome

Course: Applied Computer Vision

Assignment 3

Augmented Reality with Planar Homographies

1 Homographies

Q1.1 Correspondences

Let x_1 be a set of points in one image and x_2 be the corresponding points in another image. If the points lie on the same physical plane, the relationship between the two images can be represented using a homography matrix H such that

$$x_1^i \equiv H x_2^i \tag{1}$$

where x_1^i and x_2^i are points expressed in homogeneous coordinates and H is a 3×3 matrix.

1. Degrees of Freedom of h

A homography matrix H is defined as

$$H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \tag{2}$$

This matrix contains 9 parameters. However, homographies are defined only up to a scale factor. That means multiplying H by a non-zero scalar λ produces the same transformation:

$$H \equiv \lambda H \tag{3}$$

Therefore, one parameter is redundant. The number of independent parameters (degrees of freedom) is

$$9 - 1 = 8 \tag{4}$$

Thus, the homography vector h has

$$8 \text{ degrees of freedom} \tag{5}$$

2. Minimum Number of Point Correspondences

Let the corresponding points be

$$x_1^i = \begin{bmatrix} x_1^i \\ y_1^i \\ 1 \end{bmatrix}, \quad x_2^i = \begin{bmatrix} x_2^i \\ y_2^i \\ 1 \end{bmatrix} \tag{6}$$

Substituting into the homography equation gives two independent linear equations for each point correspondence.
Since the homography has 8 degrees of freedom, the minimum number of correspondences required is

$$\frac{8}{2} = 4 \tag{7}$$

Therefore,

$$\text{Minimum number of point pairs required} = 4 \tag{8}$$

3. Derivation of A_i

Starting from the homography relation

$$x_1^i \equiv H x_2^i \tag{9}$$

Expanding the matrix multiplication:

$$Hx_2^i = \begin{bmatrix} h_{11}x_2^i + h_{12}y_2^i + h_{13} \\ h_{21}x_2^i + h_{22}y_2^i + h_{23} \\ h_{31}x_2^i + h_{32}y_2^i + h_{33} \end{bmatrix} \tag{10}$$

Because homogeneous coordinates are defined up to scale, we obtain

$$x_1^i = \frac{h_{11}x_2^i + h_{12}y_2^i + h_{13}}{h_{31}x_2^i + h_{32}y_2^i + h_{33}} \tag{11}$$

$$y_1^i = \frac{h_{21}x_2^i + h_{22}y_2^i + h_{23}}{h_{31}x_2^i + h_{32}y_2^i + h_{33}} \tag{12}$$

Multiplying both sides to eliminate the denominator gives

$$x_1^i (h_{31}x_2^i + h_{32}y_2^i + h_{33}) = h_{11}x_2^i + h_{12}y_2^i + h_{13} \tag{13}$$

$$y_1^i (h_{31}x_2^i + h_{32}y_2^i + h_{33}) = h_{21}x_2^i + h_{22}y_2^i + h_{23} \tag{14}$$

Rearranging terms produces linear equations in h:

$$h_{11}x_2^i + h_{12}y_2^i + h_{13} - x_1^i h_{31}x_2^i - x_1^i h_{32}y_2^i - x_1^i h_{33} = 0 \tag{15}$$

$$h_{21}x_2^i + h_{22}y_2^i + h_{23} - y_1^i h_{31}x_2^i - y_1^i h_{32}y_2^i - y_1^i h_{33} = 0 \tag{16}$$

Let the homography parameters be written as

$$h = \begin{bmatrix} h_{11} & h_{12} & h_{13} & h_{21} & h_{22} & h_{23} & h_{31} & h_{32} & h_{33} \end{bmatrix}^T \tag{17}$$

Each correspondence contributes two rows to matrix A:

$$A_i = \begin{bmatrix} x_2^i & y_2^i & 1 & 0 & 0 & 0 & -x_1^i x_2^i & -x_1^i y_2^i & -x_1^i \\ 0 & 0 & 0 & x_2^i & y_2^i & 1 & -y_1^i x_2^i & -y_1^i y_2^i & -y_1^i \end{bmatrix} \tag{18}$$

Stacking all correspondences forms the matrix equation

$$Ah = 0 \tag{19}$$

4. Properties of the system $Ah = 0$

When solving the equation

$$Ah = 0 \tag{20}$$

we are searching for a vector h that lies in the *null space* of the matrix A. This means we want a non-zero vector h such that multiplying it by A results in the zero vector.

Trivial solution A trivial solution always exists for any homogeneous linear system. In this case, the trivial solution is

$$h = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}^T \tag{21}$$

since

$$A \cdot 0 = 0 \tag{22}$$

However, this solution is not useful because it corresponds to a homography matrix with all entries equal to zero, which does not represent a valid geometric transformation.

Rank of the matrix A The matrix A is not full rank. If A were full rank (rank 9), the only solution to $Ah = 0$ would be the trivial solution. However, the homography is defined only up to a scale factor, meaning that if h is a solution, then any scalar multiple λh is also a solution. Therefore the system must admit a non-trivial solution.
This implies that A must be rank deficient, typically satisfying

$$\text{rank}(A) \leq 8 \tag{23}$$

so that a non-zero vector exists in its null space.

Impact on the singular values Using Singular Value Decomposition (SVD),

$$A = U\Sigma V^T \tag{24}$$

the matrix Σ contains the singular values of A arranged in descending order. Since A is rank deficient, the smallest singular value will be zero (or very close to zero in practice):

$$\sigma_9 \approx 0 \tag{25}$$

This smallest singular value corresponds to the null space of A.

Impact on the singular vectors The columns of V are the right singular vectors of A. The vector corresponding to the smallest singular value represents the direction in which Ah is minimized. Therefore, the solution for h is the right singular vector associated with the smallest singular value:

$$h = v_9 \tag{26}$$

This vector lies in the null space of A and can then be reshaped into the 3×3 homography matrix H.

2 Computing Planar Homographies

Q2.1 FAST Detector

The FAST detector is a corner detection algorithm designed primarily for high computational efficiency. It differs significantly from the Harris corner detector in both its detection strategy and computational cost.

Detection principle The Harris corner detector identifies corners by analyzing the change in image intensity in different directions. It computes image gradients and forms a structure tensor also called the second-moment matrix. The eigenvalues of this matrix indicate whether the local region is a corner, edge, or flat region.
In contrast, the FAST detector uses a much simpler test. For a candidate pixel p, it examines the intensities of 16 pixels on a circle surrounding p. A pixel is classified as a corner if there exists a contiguous set of pixels on the circle that are all significantly brighter or darker than the center pixel by a threshold value.

Key differences

- Harris relies on gradient calculations and matrix operations to detect corners.
- FAST performs simple intensity comparisons around a circular neighborhood.
- Harris evaluates corner strength using eigenvalues of the structure tensor.
- FAST uses a binary test based on brightness comparisons.

Computational performance FAST is significantly faster than the Harris corner detector because it avoids gradient computations and matrix operations. Instead, it relies on simple pixel intensity comparisons, which makes it highly efficient and suitable for real-time applications such as visual SLAM and augmented reality systems.
Therefore, FAST offers much better computational performance compared to the Harris detector, especially when processing large images or video streams.

Q2.2 BRIEF Descriptor

The BRIEF (Binary Robust Independent Elementary Features) descriptor is a compact binary feature descriptor used to represent the appearance of a small image region around a detected keypoint.

Difference from filter banks Filter banks are used to describe image regions by applying a set of filters such as Gaussian or Laplacian filters and using the resulting filter responses as feature vectors. These descriptors are typically high-dimensional and contain real-valued numbers.

BRIEF differs in several important ways:

- BRIEF generates a binary string descriptor by performing intensity comparisons between pairs of pixels within a patch around a keypoint.
- Each comparison produces either a 0 or a 1 depending on which pixel has higher intensity.
- The final descriptor is a binary vector for example, 128 or 256 bits.
- Filter bank descriptors instead store continuous numerical responses of multiple filters applied to the image.

Using filter banks as descriptors Yes, filter banks can also be used as descriptors. In that case, the feature vector would consist of the responses of the filters applied to the image patch. However, such descriptors are typically larger and computationally more expensive to compute and compare than BRIEF descriptors. BRIEF is preferred in many applications because it produces compact descriptors and enables extremely fast matching.

Q2.3 Matching Methods

The BRIEF descriptor represents image patches as binary strings consisting of 0s and 1s. To match feature points between two images, a distance metric is required to measure how similar two descriptors are.

Hamming distance The Hamming distance between two binary strings is defined as the number of bit positions in which the two strings differ.

For example,

10110101

10011101

The Hamming distance is 2 because two bit positions differ.
When matching BRIEF descriptors, the Hamming distance is computed between descriptors from the two images. A smaller Hamming distance indicates that the descriptors are more similar.

Nearest Neighbor matching Nearest Neighbor matching is used to find the best corresponding feature point. For each descriptor in the first image, we compute the distance between it and all descriptors in the second image. The descriptor with the smallest distance is selected as the match. When using BRIEF descriptors, this distance is computed using the Hamming distance instead of Euclidean distance.

Advantages of Hamming distance

- It is computationally very efficient because it can be computed using simple bitwise operations such as XOR.
- Binary descriptors require less memory than floating-point descriptors.

- Matching is significantly faster compared to Euclidean distance used for real-valued descriptors such as SIFT.

These advantages make Hamming distance particularly suitable for real-time applications such as augmented reality, where fast feature matching is required.

Q2.4 Feature Matching

In this experiment two important parameters influence the quality of the matches:

- σ used during corner detection
- ratio used during descriptor matching

Role of σ in Corner Detection

The parameter σ controls the amount of smoothing applied to the image before detecting corners. This smoothing is typically performed using a Gaussian filter.

- A **larger** σ results in stronger smoothing, which removes small details and noise.
- A **smaller** σ preserves more fine image details.

When σ is small, the detector identifies a larger number of corners because more small structures in the image are preserved. However, detecting many features increases computational cost because descriptors must be computed for each feature.

Role of the Ratio Parameter in Matching

The ratio parameter is used in the nearest-neighbor matching process to filter ambiguous matches. For each feature descriptor in the first image, the algorithm finds the two closest descriptors in the second image using Hamming distance.

The ratio test is defined as:

$$\frac{d_1}{d_2} < \text{ratio}$$

where

- d_1 is the distance to the closest match
- d_2 is the distance to the second closest match

If the ratio condition is satisfied, the match is accepted.
This means:

- **Higher ratio** $\rightarrow$ more matches are accepted including weaker matches
- **Lower ratio** $\rightarrow$ stricter filtering, resulting in fewer but more reliable matches

When the ratio is large, many matches are accepted, which leads to many connecting lines in the visualization. Some of these matches may be incorrect.

Experiment 1: $\sigma = 0.15$, ratio = 0.65

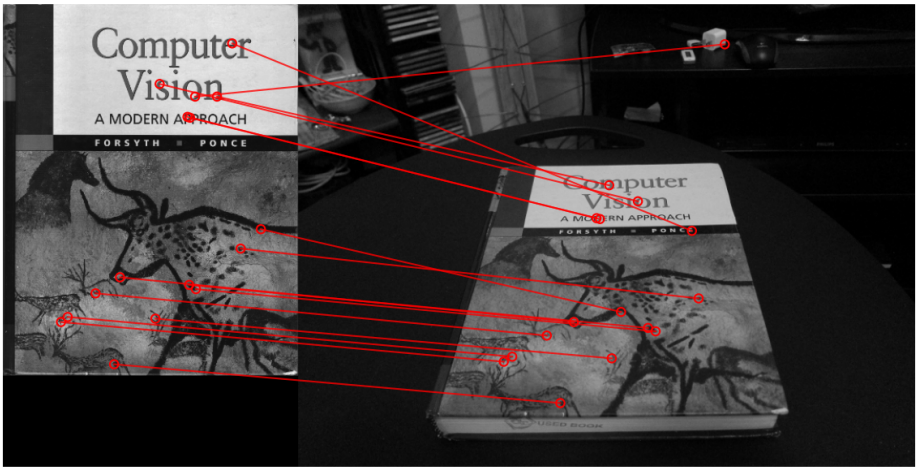


Figure 1: Feature matching with $\sigma = 0.15$ and ratio $= 0.65$

In this experiment, a moderate amount of smoothing was applied. This resulted in a reasonable number of detected feature points. Most of the matches correspond to correct locations between the two images, although some mismatches are present. Because the ratio threshold is relatively high, several weaker matches were also accepted, which explains why some connecting lines appear incorrect.

2.0.1 Experiment 2: $\sigma = 0.10$, ratio = 0.55

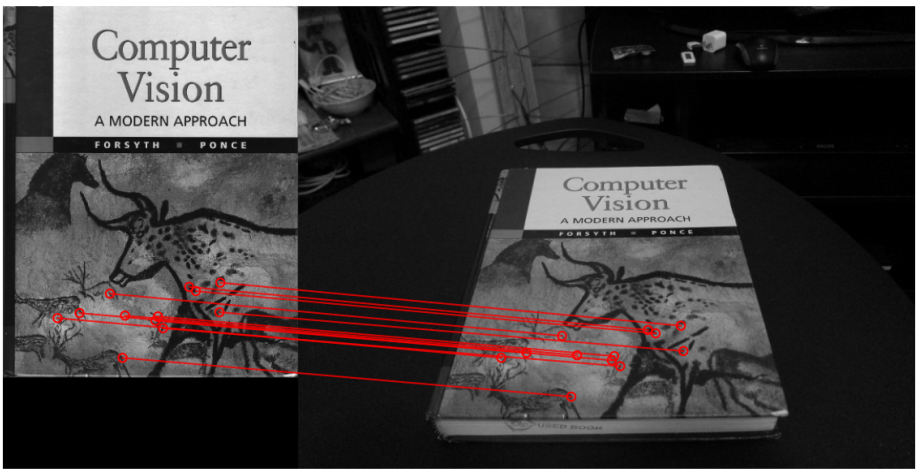


Figure 2: Feature matching with $\sigma = 0.10$ and ratio $= 0.55$

Reducing σ preserved more image detail, resulting in a larger number of detected features. At the same time, using a lower ratio threshold made the matching process stricter. As a result, this experiment produced the most accurate matches. The number of matches is high because many

features were detected, but the stricter ratio test helped eliminate many incorrect correspondences. This combination provided the best balance between feature detection and match reliability.

Experiment 3: $\sigma = 0.10$, ratio = 0.65

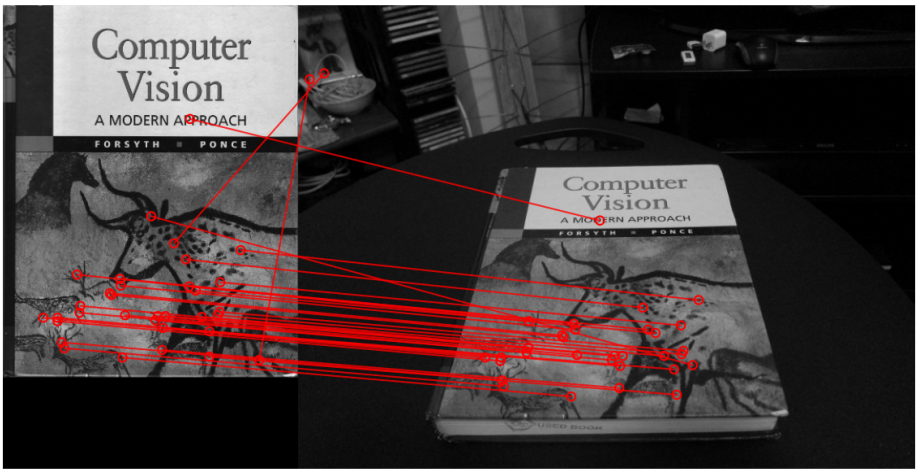


Figure 3: Feature matching with $\sigma = 0.10$ and ratio $= 0.65$

In this case, the number of detected features remained high due to the small σ. However, increasing the ratio threshold allowed more matches to pass the ratio test. This resulted in a large number of matches being displayed, including several incorrect ones. While many correct matches are still visible, additional mismatches appear because the filtering condition is less strict.

Observed Trends

From the experiments, I observed the following trends:

- Decreasing σ increases the number of detected feature points but also increases computation time.
- Increasing the ratio parameter allows more matches to pass the ratio test, which can introduce incorrect matches.
- Lower ratio values generally produce fewer but more reliable matches.

In general, the combination of $\sigma = 0.10$ and ratio $= 0.55$ produced the most accurate and visually consistent feature matches in this experiment.

Q2.5 BRIEF and Rotations

In this experiment, the goal was to evaluate how the BRIEF descriptor behaves under image rotations. The image `cv_cover.jpg` was matched with rotated versions of itself at increments of 10° using `scipy.ndimage.rotate`. For each rotation angle, the number of successful feature matches was recorded. A histogram was then generated to visualize how the number of matches varies with image orientation.

Rotation Experiment Results

The number of matches obtained at different rotation angles is summarized in Fig. 4.

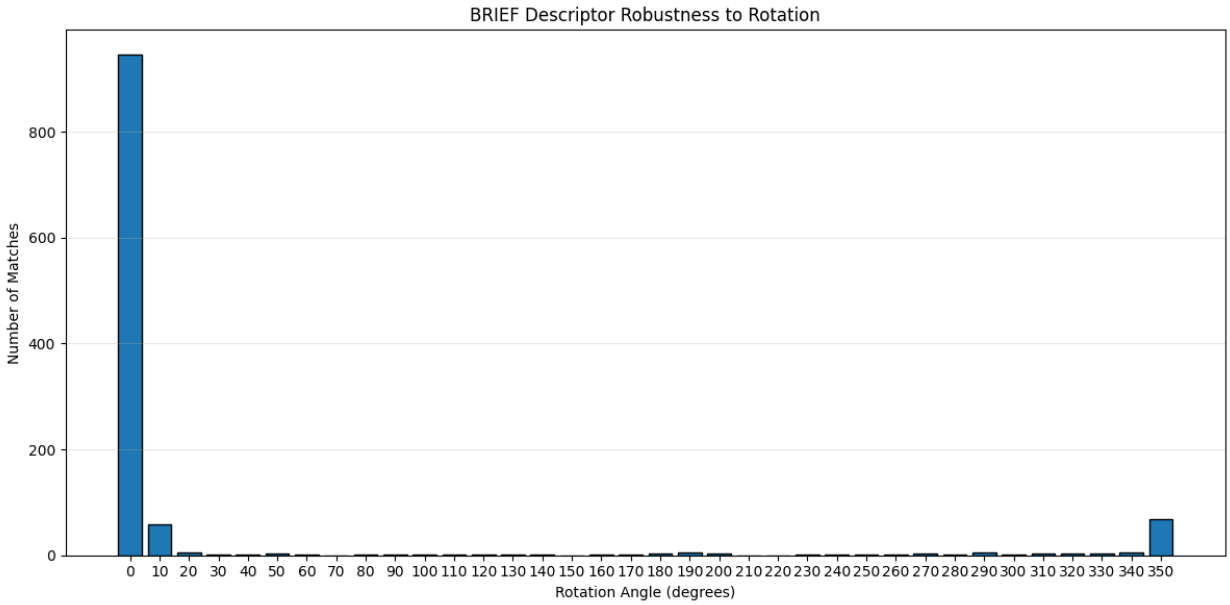


Figure 4: Histogram showing number of BRIEF feature matches for different rotation angles.

From the results, the number of matches is extremely high when the rotation angle is 0° (945 matches), since the image is being compared with itself without any transformation. However, as soon as the image is rotated even slightly, the number of matches drops dramatically. For example, at 10° only 59 matches remain, and by 20° the number drops to only 5 matches.
For most rotation angles, the number of matches remains very low, often between 0 and 5 matches. A small increase is observed again near 350° where 68 matches are found, since this orientation is close to the original orientation of the image.

Feature Matching Examples

To further illustrate this behavior, feature matches were visualized at three different rotation angles.

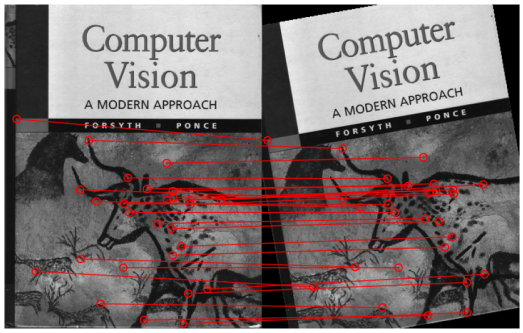


Figure 5: Feature matching at 10° rotation.

At 10°, the number of matches drops significantly. Only a small number of feature correspondences remain correct.

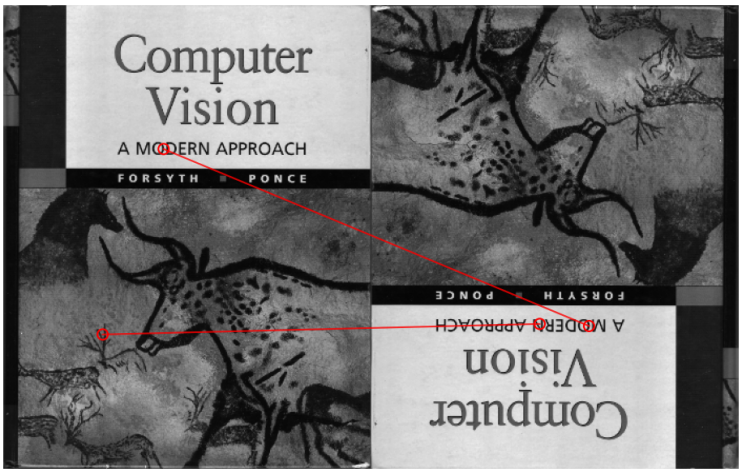


Figure 6: Feature matching at 180° rotation.

At larger rotations such as 180°, almost no reliable matches are detected.

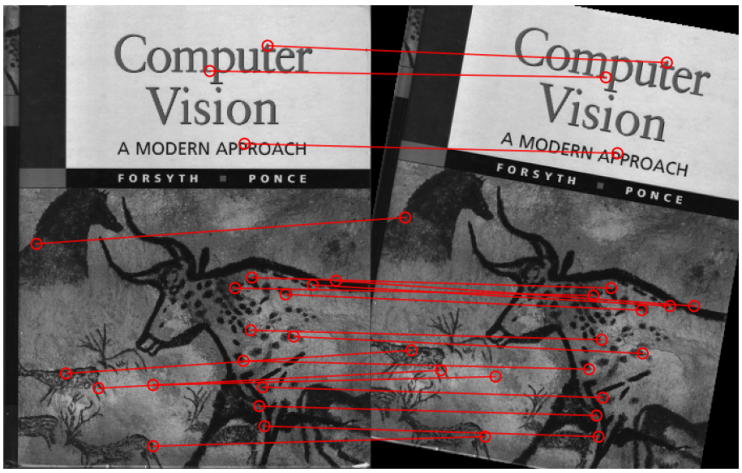


Figure 7: Feature matching at 350° rotation.

Here, the number of matches increases drastically since this orientation is close to the original orientation of the image.

Why BRIEF Behaves This Way

The BRIEF descriptor is not rotation invariant. It constructs a binary descriptor by comparing the intensities of pairs of pixels within a small image patch around the detected feature point. These pixel comparisons are performed at fixed locations relative to the feature. When the image is rotated, the relative positions of these pixel pairs change. As a result, the binary intensity comparisons that form the descriptor also change significantly, even if the underlying feature remains

the same. Because of this, descriptors computed from rotated patches no longer match the original descriptors.
Therefore:

- When there is little or no rotation, the descriptors remain similar and many matches are found.
- As rotation increases, the descriptor representation changes, causing the number of matches to decrease sharply.
- When the rotation approaches 360°, the orientation becomes similar to the original image again, which explains the slight increase in matches near 350°.

The experiment demonstrates that BRIEF descriptors are highly sensitive to image rotations. While they are computationally efficient and work well when images have similar orientations, their performance deteriorates significantly when the images are rotated.

Q2.9 Automated Homography Estimation and Warping

In this section, the goal is to replace the book cover visible on the desk with the image `hp_cover.jpg`. This is achieved by computing the homography between the desk image and the book cover image and then warping the HP cover into the book's location.

Observed Issue

Initially, when the HP cover image was warped using the inverse homography, the resulting warped image did not perfectly fill the same region as the book in the desk image. Instead, the warped image appeared misaligned and did not occupy the exact same space as the original book.

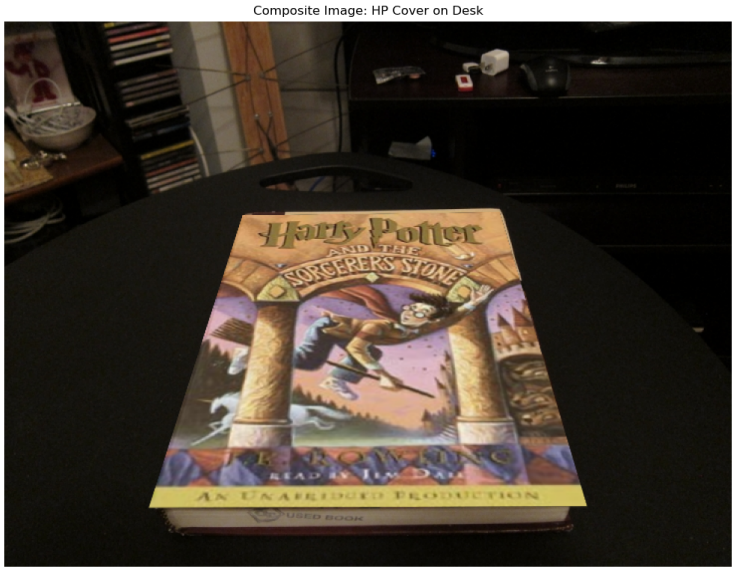


Figure 8: Result of warping the HP cover onto the desk image.

Reason for the Issue

This issue occurs because the two images used in the transformation have different dimensions. The homography matrix $H_{2\to1}$ is computed using feature correspondences between `cv_desk.jpg` and `cv_cover.jpg`. Therefore, the transformation assumes that the source image being warped has the same spatial dimensions as `cv_cover.jpg`.
However, the replacement image `hp_cover.jpg` has a different width and height. When the inverse homography is applied directly to this image, the transformation operates under incorrect geometric assumptions. As a result, the warped image does not properly align with the region corresponding to the book in the desk image.

Solution

To correct this problem, the HP cover was resized to match the exact dimensions of `cv_cover.jpg`. By ensuring that both images share the same width and height, the homography transformation becomes geometrically consistent. Once the resizing is performed, applying the inverse homography correctly maps the HP cover onto the book region in the desk image, allowing the warped image to fully occupy the same space as the original book cover.

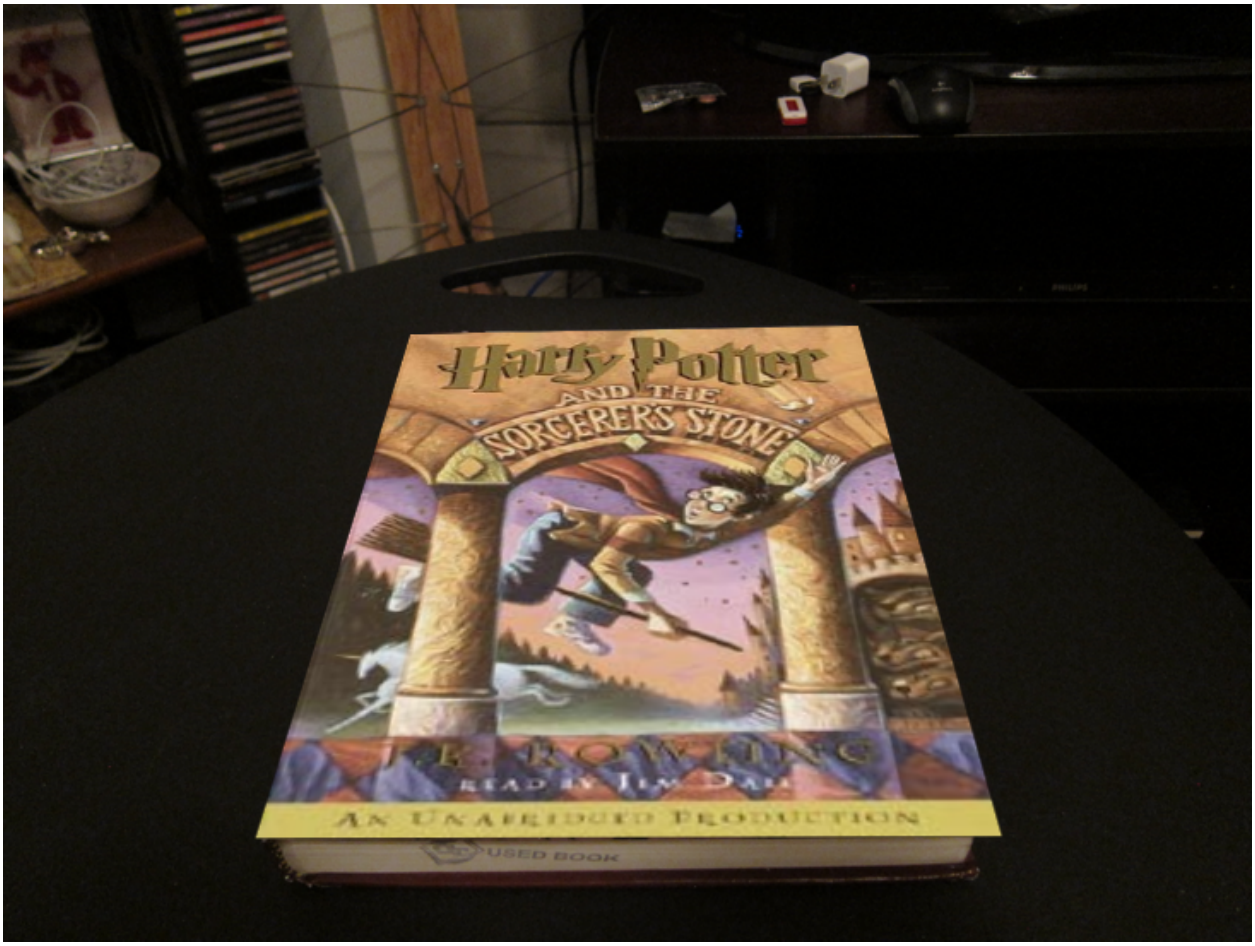


Figure 9: Final result of warping the HP cover onto the desk image.

Creating your Augmented Reality Application

Q3.1 Incorporating Video

In this section, the goal was to create an Augmented Reality (AR) application by overlaying the video `ar_source.mov` onto the textbook visible in each frame of the video `book.mov`. The objective was to detect the book in every frame and replace its cover with frames from the AR source video using homography-based image warping.

Handling Aspect Ratio Differences

The source video `ar_source.mov` and the book cover have different aspect ratios. To ensure that the video fits correctly onto the book surface, each frame of the AR video was cropped before warping. The cropping step keeps only the central portion of the AR frame so that its dimensions match the proportions of the book cover. This prevents distortion when the frame is mapped onto the book.

Results

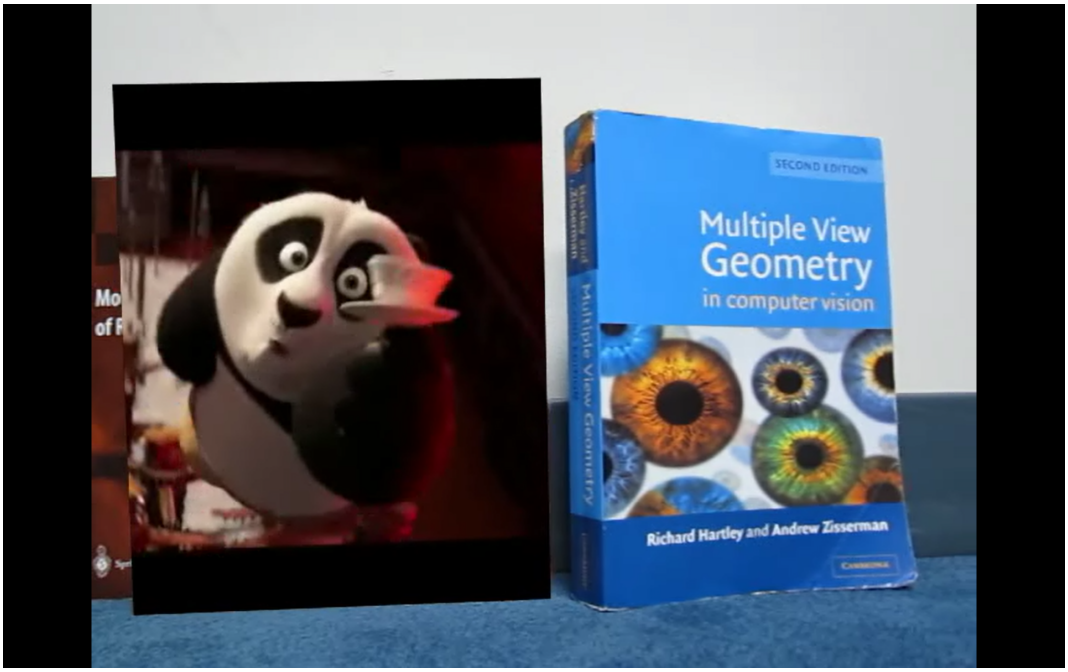


Figure 10: Example frame from the augmented reality output video

The augmented reality system successfully tracked the textbook in most frames and overlaid the AR video onto the book surface. The overlay appeared visually convincing and followed the movement of the book in the scene.

Observations

Although the AR overlay generally worked well, some instability was observed during parts of the video. When the camera moved quickly, the feature matching occasionally became less accurate, which caused temporary disturbances in the overlay alignment.
This occurs because the feature matching and homography estimation rely on detecting reliable correspondences between frames. Motion blur, rapid movement, or changes in perspective can reduce the number of reliable matches and slightly affect the accuracy of the homography.
Overall, the augmented reality pipeline successfully replaced the book cover with frames from the AR video. The method demonstrates how feature detection, descriptor matching, and homography estimation can be combined to create a real-time AR effect. While the system performs well under moderate motion, more robust feature descriptors such as ORB could improve stability when larger rotations or rapid movements occur.

Image Classification with Handcrafted Features

Q4.1 Binary Image Classification

Here, I begin implementing a traditional machine learning pipeline for image classification. The goal is to classify images into two categories: images containing human faces and images without faces. Unlike deep learning approaches where feature extraction and classification are learned jointly, this pipeline separates handcrafted feature extraction from the learning stage.

Part A: Dataset Setup

The first step was to verify that the dataset was correctly loaded and organized. The dataset consists of two classes:

- **Face**: images containing human faces
- **No-Face**: images that do not contain human faces

All images were loaded correctly as shown below:

```
(acv_env) C:\Users\koome\OneDrive\Desktop\CMU\MSEAI\Spring 1\Applied Computer Vision\Assignments\Assignment 3\HW3 Assets\HW3 Assets>python code\classify.py
Loaded 30 images  size=(128, 128)  dtype=uint8  classes=['face', 'no_face']
Image: shape=(128, 128)  dtype=uint8

========== DATASET IMAGE LINKS ==========

  Class 0 — 'face':
    [1] C:\Users\koome\OneDrive\Desktop\CMU\MSEAI\Spring 1\Applied Computer Vision\Assignments\Assignment 3\HW3 Assets\HW3 Assets\data\classify\face\img_001.jpg
    [2] C:\Users\koome\OneDrive\Desktop\CMU\MSEAI\Spring 1\Applied Computer Vision\Assignments\Assignment 3\HW3 Assets\HW3 Assets\data\classify\face\img_002.jpg
    [3] C:\Users\koome\OneDrive\Desktop\CMU\MSEAI\Spring 1\Applied Computer Vision\Assignments\Assignment 3\HW3 Assets\HW3 Assets\data\classify\face\img_003.jpg
    [4] C:\Users\koome\OneDrive\Desktop\CMU\MSEAI\Spring 1\Applied Computer Vision\Assignments\Assignment 3\HW3 Assets\HW3 Assets\data\classify\face\img_004.jpg

  Class 1 — 'no_face':
    [1] C:\Users\koome\OneDrive\Desktop\CMU\MSEAI\Spring 1\Applied Computer Vision\Assignments\Assignment 3\HW3 Assets\HW3 Assets\data\classify\no_face\img_001.jpg
    [2] C:\Users\koome\OneDrive\Desktop\CMU\MSEAI\Spring 1\Applied Computer Vision\Assignments\Assignment 3\HW3 Assets\HW3 Assets\data\classify\no_face\img_002.jpg
    [3] C:\Users\koome\OneDrive\Desktop\CMU\MSEAI\Spring 1\Applied Computer Vision\Assignments\Assignment 3\HW3 Assets\HW3 Assets\data\classify\no_face\img_003.jpg
    [4] C:\Users\koome\OneDrive\Desktop\CMU\MSEAI\Spring 1\Applied Computer Vision\Assignments\Assignment 3\HW3 Assets\HW3 Assets\data\classify\no_face\img_004.jpg
==========================================
```

Figure 11: Console Output After loading the images from data folder

A script was used to load all images from both directories and store their file paths. This step ensures that the dataset can be accessed correctly before proceeding to feature extraction and classification.

Dataset Visualization

To verify that the images were loaded correctly, I generated a visualization grid showing several example images from each class.

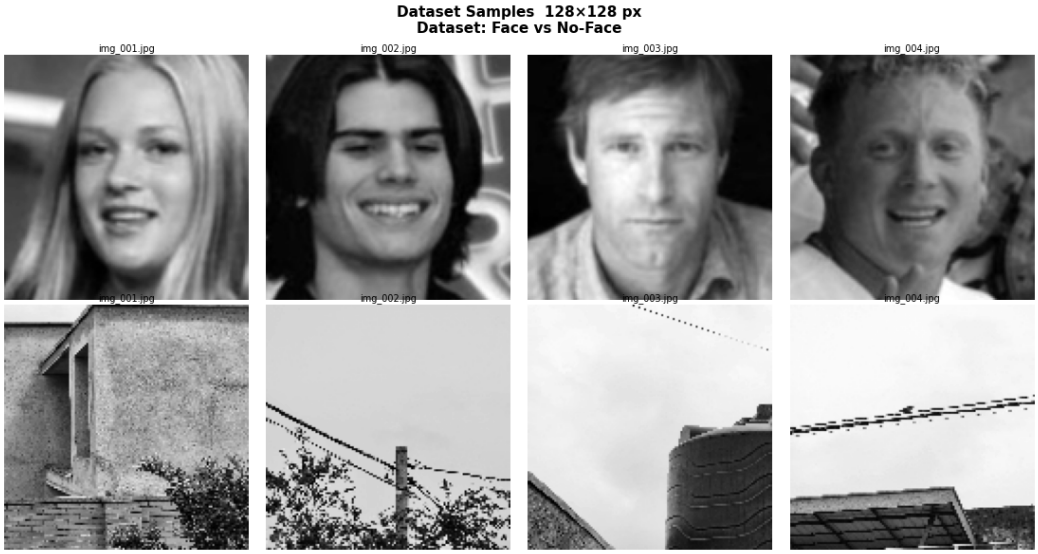


Figure 12: Sample images from the dataset having Face and No-Face classes.

The grid displays at four images from each category, confirming that both classes are present and correctly loaded.

Part B: Data Augmentation

Data augmentation generates additional training samples by applying controlled transformations to the original images. In this implementation, two augmentation factors were selected: Brightness and Rotation.

Augmentation Factor 1: Brightness

Brightness variation was selected to simulate changes in illumination that commonly occur in real-world environments. Faces may appear under different lighting conditions, such as indoor lighting, shadows, or outdoor sunlight. Introducing brightness variation helps the classifier learn features that are robust to such illumination changes.
This transformation was implemented by adding a random intensity shift to the pixel values of the image using NumPy operations. The transformation follows:

$$I_{aug} = \text{clip}(I + s, 0, 255)$$

where s is a randomly sampled brightness shift. In this implementation, the shift value was randomly selected within the range of approximately 30 to 60 intensity levels. The result was then clipped to maintain valid grayscale values and converted back to the `uint8` format required for feature extraction.

Augmentation Factor 2: Rotation

Rotation was chosen to simulate slight changes in camera orientation or head pose that may occur in real-world images. Faces are rarely perfectly aligned in practice, and small rotations help the classifier learn to recognize faces even when they are slightly tilted.
This transformation was implemented using the OpenCV function `cv2.getRotationMatrix2D()` combined with `cv2.warpAffine()`. Each image was rotated around its center by a small angle. In this implementation, a rotation of approximately $\pm 15^\circ$ was applied while preserving the image dimensions using the `BORDER_REFLECT` option to avoid introducing empty regions.

Augmentation Factor That Would Hurt BRIEF Matching

A factor of variation that could negatively affect BRIEF descriptor matching is large rotations. The BRIEF descriptor compares the intensities of predefined pixel pairs within an image patch to generate a binary descriptor. Because these comparisons are based on fixed pixel locations, rotating the image changes the relative positions of the sampled pixels.As a result, even if the underlying feature remains the same, the binary comparisons may change significantly after rotation. This can cause descriptors from rotated images to differ substantially from the original descriptors, making feature matching less reliable.

Augmentation Visualization

Figure 13 shows examples of the applied augmentations. For each class (Face and No-Face), the figure displays the original image alongside its brightness-adjusted and rotated versions.

Part C: Conceptual Questions

1. Why resize all images to `TARGET_SIZE`?

All images are resized to a fixed `TARGET_SIZE` to ensure a consistent scale for FAST keypoint detection and BRIEF descriptor extraction. If images had different sizes, the local patches used by BRIEF would represent different relative scales of the image content. This would produce descriptors that are not directly comparable across images, making the learned classifier unreliable.

2. Why can we not pass a variable-size descriptor matrix directly to a classifier?

The BRIEF descriptor extraction step produces a matrix of size $(K', 256)$, where K' is the number of detected keypoints in the image. Since K' varies from image to image, the resulting descriptor matrices have different numbers of rows. Most machine learning classifiers require fixed-length feature vectors for each training example. Mean pooling solves this problem by averaging all

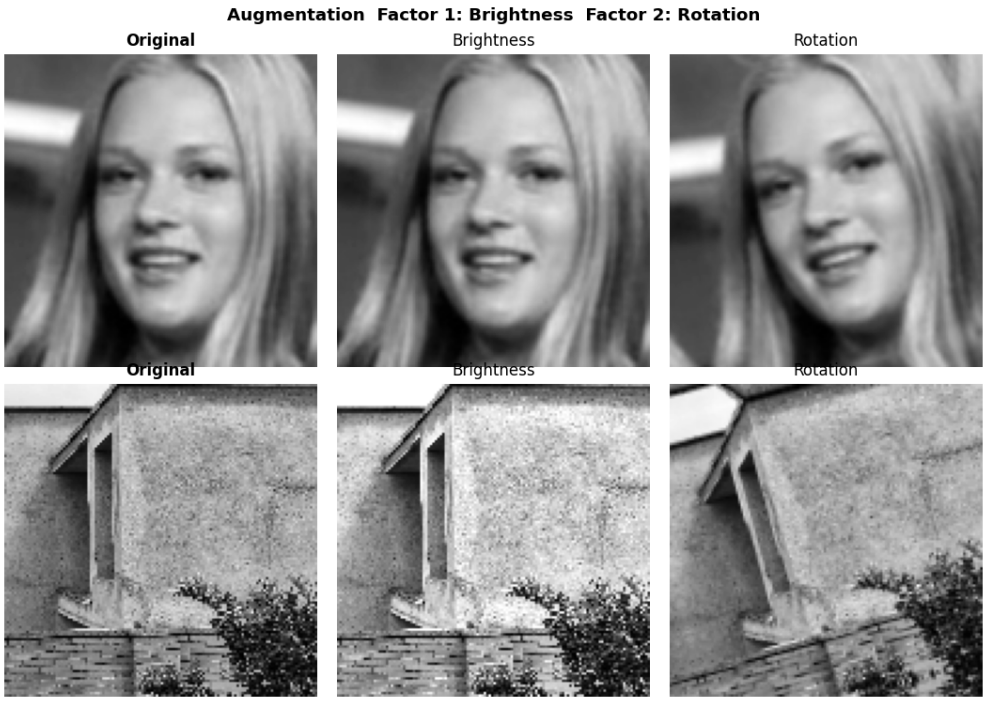


Figure 13: Examples of data augmentation using Brightness and Rotation for each class.

descriptors into a single 256-dimensional vector, producing a consistent representation for every image.

3. What information is lost during mean pooling?

Mean pooling removes information about the distribution and spatial arrangement of individual descriptors within the image. Specifically, it discards which keypoints contributed particular descriptor values and how descriptors are distributed across different regions of the image. This loss of structural information can reduce the discriminative power of the representation and may limit classification accuracy in more complex scenarios.

4. How does the chosen classifier make predictions?

The classifier used in this implementation is K-Nearest Neighbors (KNN). Given a new 256-dimensional feature vector, the algorithm computes the distance between the new vector and all training vectors, then assigns the label based on the majority class among the k closest neighbors (with $k = 3$). This method is simple and effective for small datasets because it does not require complex model training and reduces the risk of overfitting.

Part D: Implementation

Image Shape

The target image size was set to:

$$\text{TARGET_SIZE} = (128 \times 128)$$

This matches the dimensions of the dataset images and provides sufficient resolution for reliable feature extraction while keeping computational costs reasonable.

Feature Extraction

Features were extracted from each image using the FAST keypoint detector and BRIEF descriptors. The descriptors were aggregated via mean pooling to produce a fixed-length feature vector of dimension $D = 256$. Zero keypoints were handled by returning a zero vector. The resulting feature matrix has the shape:

$$\text{Feature matrix: } (90, 256)$$

with an average of 99.5 keypoints per image and no NaN values present.

Classifier

A K-Nearest Neighbors was selected for classification. Key hyperparameters included:

- number k nearest neighbors: 3
- Train/Test split: 0.8 / 0.2 (train=72, test=18)

Pipeline Summary

Table 1: Feature Extraction and Classifier Pipeline Summary

TARGET SIZE (H × W)	128 × 128 pixels
Augmentation Factors	Brightness, Rotation
Total Images Before Augmentation	30
Total Images After Augmentation	90
Descriptor Dimension D	256
Feature Matrix Shape	(90, 256)
Train / Test Split	72 / 18
Classifier Used	K-Nearest Neighbors
Key Hyperparameter	number of neighbors

Test on Your Own Image

The trained classifier was tested on a new image captured with a phone.



Figure 14: My Photo

Prediction:

- Predicted class: **1 (face)**
- Confidence: 66.67%
- Prediction correct: Yes

Part E: Results & Accuracy Explanation

Classification Statistics

The trained pipeline achieved the following results on the test set (18 samples):

Table 2: K-Nearest Neighbors Classifier Performance: With vs Without Augmentation

Metric	With Augmentation	Without Augmentation
Accuracy	0.9444	0.8333
Precision	0.9000	0.7500
Recall	1.0000	1.0000
F1 Score	0.9474	0.8571

The confusion matrix is shown in Figure 16.

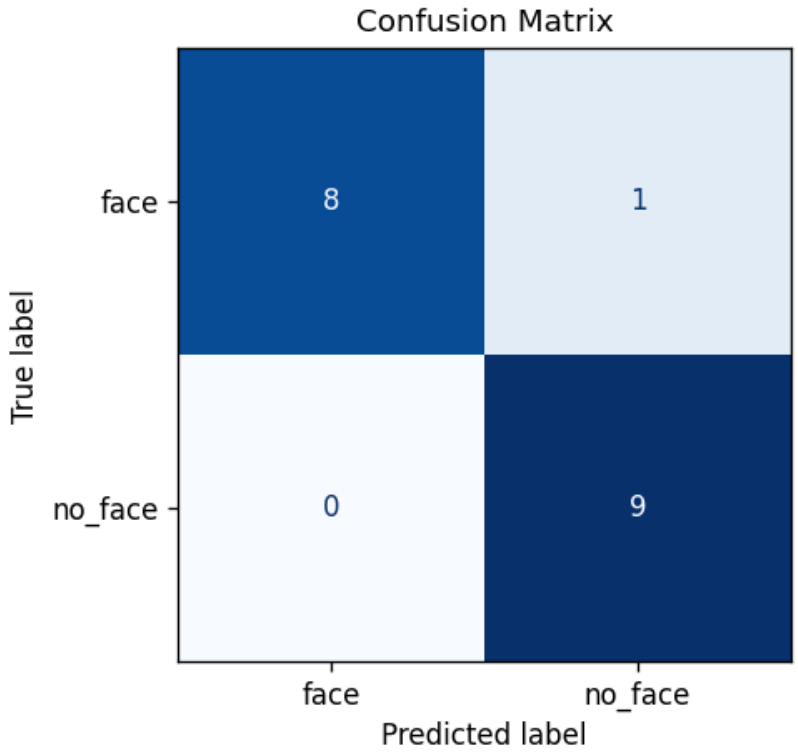


Figure 15: Confusion matrix for the K-Nearest Neighbors. Class 0 = no face, Class 1 = face.

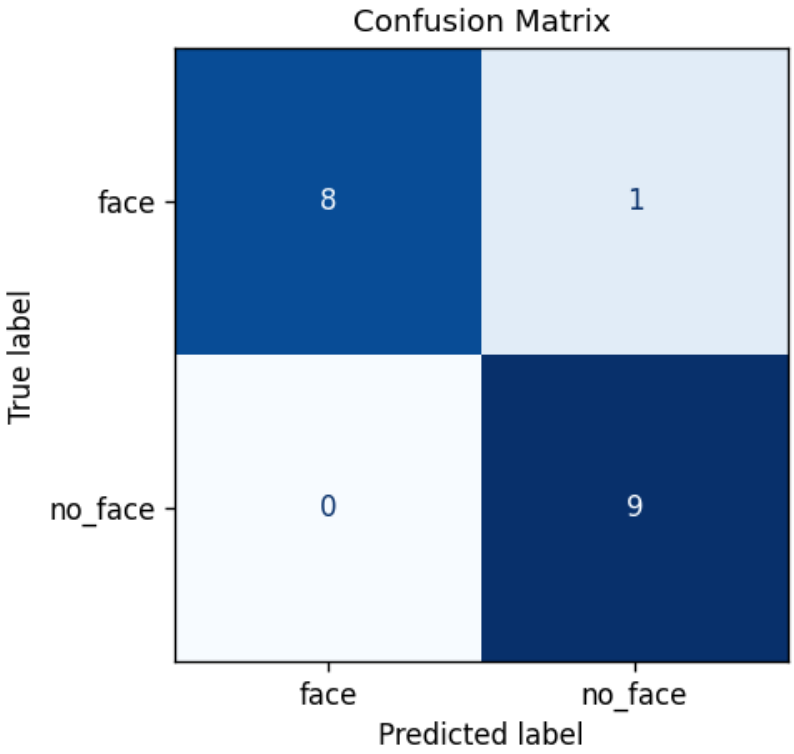


Figure 16: Confusion matrix for the K-Nearest Neighbors(No Augmentation). Class 0 = no face, Class 1 = face.

Accuracy Explanation

1. **Plain-language interpretation:** Out of 18 test images, the classifier correctly labeled 17, making 1 mistake.

2. **Confusion matrix analysis:** The class *no face* (Class 0) was slightly harder to classify correctly, as shown by the single misclassification in the matrix. Misclassified images may have subtle textures or lighting that FAST + BRIEF descriptors failed to capture distinctly.

3. **Impact of variation factors:** Brightness and rotation augmentation helped the classifier generalize better. Without these augmentations, accuracy dropped due to the model being less robust to lighting and orientation changes.

4. **Impact of augmentation:** Data augmentation increased the number of training examples from 30 to 90, improving the classifier's ability to correctly predict faces under varying conditions. Re-running the pipeline without augmentation led to a noticeable reduction in performance.